Figure 1: The memory structure of Heap

to the beginning of the Old space. Next comes the Permanent Generation, a.k.a. PermGen, which existed prior to Java 8 and was used to store the metadata information of the objects. With Java 8, the class definitions are now loaded into metaspace, which is located in the native memory and does not interfere with the regular Heap objects. By default, the metaspace size is only limited by the amount of native memory available to the Java process.

Moving on, GC is mainly categorised into three types, namely, Minor GC, Major GC and Full GC. Minor GC works on the Young generation and is triggered when the JVM is unable to allocate the space for a new object. Besides, it does trigger the stop-the-world pause suspending all the live threads. Major GC runs on the Old Generation while the full GC works on cleaning the entire Heap.

Java implements different algorithms to achieve GC and, to understand them, let's start with some basic terminology.

- **Marking reachable objects:** GC defines some specific objects as GC roots. Examples are local variables and input parameters of currently executing methods, active threads, static field of the loaded classes, JNI references, etc. This phase calls the stop-the-world pause.
- **Removing unused objects:** Removal of unused objects is somewhat different for different GC algorithms, but all such algorithms can be divided into three groups—sweeping, compacting and copying.

Sweeping or 'mark and sweep' means that after the marking phase is complete, all space occupied by unvisited objects is considered free and can thus be reused to allocate new objects.

Figure 2: Marking live objects in the Heap

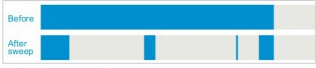

Figure 3: Memory status after sweep

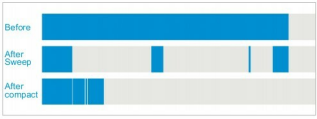

Figure 4: Memory status after compacting

Compact or mark-sweep-compact solves the shortcomings of 'mark and sweep' by moving all marked – and thus alive – objects to the beginning of the memory region.

Copy or 'mark and copy' algorithms are very similar to the 'mark and compact' as they too relocate all live objects. The important difference is that the target of relocation is a different memory region as a new home for survivors.

Figure 5: Memory status after copying

Young	Tenured	JVM options
Incremental	Incremental	-Xincgc
Serial	**Serial**	**-XX:+UseSerialGC**
Parallel Scavenge	Serial	-XX:+UseParallelGC -XX:-UseParallelOldGC
Parallel New	Serial	N/A
Serial	Parallel Old	N/A
Parallel Scavenge	**Parallel Old**	**-XX:+UseParallelGC -XX:+UseParallelOldGC**
Parallel New	Parallel Old	N/A
Serial	CMS	-XX:-UseParNewGC -XX:+UseConcMarkSweepGC
Parallel Scavenge	CMS	N/A
Parallel New	**CMS**	**-XX:+UseParNewGC -XX:+UseConcMarkSweepGC**
G1		**-XX:+UseG1GC**

Figure 6: Different algorithms for garbage collection

We are done with all the basics and the terminology. Let's now explore the different algorithms of GC. Basically, these can be put in four categories—namely, Serial, Parallel, CMS and G1.

Serial GC: In this, the GC uses mark-copy for the Young Generation and mark-sweep-compact for the Old Generation. Both the collectors are single threaded and trigger stop-the-world pauses, stopping all application threads. This GC algorithm cannot thus take advantage of multiple CPU cores commonly found in modern hardware.

Parallel GC: This uses mark-copy in the Young Generation and mark-sweep-compact in the Old Generation. Both Young and Old collections trigger stop-the-world events, stopping all application threads from performing garbage collection. It uses multiple threads and, thus, is suitable on multi-core machines in cases where your primary goal is to increase throughput. High throughput is achieved due to effective usage of system resources in two ways.

Continued on Page...63